jobwatch

A job-postings watcher for international students
Draft v0.1, request for scope review

Alex Tran

July 19, 2026

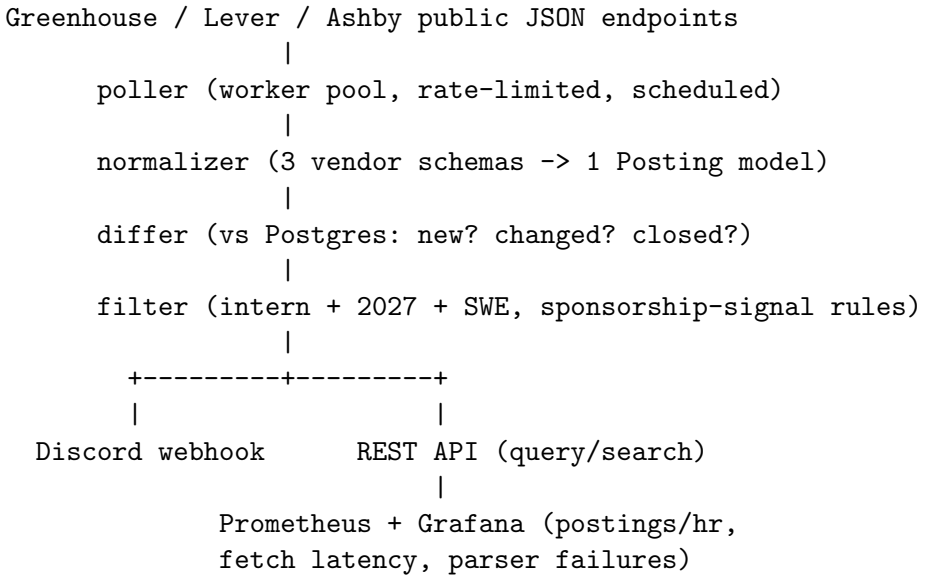
Status: pre-build draft. Nothing is written yet. I am asking for a rating and a scope check before I start.

Problem

Summer 2027 internship postings go live across hundreds of company career pages starting August 2026. Today I find them by scrolling job boards and a community GitHub repo, manually, daily. Worse: as an international student (F-1), a large share of postings are citizenship-gated (ITAR, "US persons only", "unable to sponsor"), and I usually discover that only after reading deep into the JD or after applying. Every international friend I have hits the same wall.

What it is

A Go service that watches the career boards of a configurable list of companies and posts to a Discord channel when a new Summer 2027 SWE internship appears, with a flag for sponsorship risk. Discord is the entire user interface. There is no web frontend, by design.



Why not just use Simplify or existing job bots

1. Sponsorship-signal detection. No mainstream tool flags "this posting smells citizenship-gated" for F-1 students. That filter is the point.
2. Closed-posting diffs. I want to know when a posting disappears, not only when it appears.

3. Per-user filters when friends join (keywords, companies, sponsorship strictness).
4. Honestly: the point is building it. This is my backend/infra learning project and I will be user #1 daily from August.

Data sources

Public, documented JSON endpoints only. No scraping, no LinkedIn.

- Greenhouse: `boards-api.greenhouse.io/v1/boards/{company}/jobs`
- Lever: `api.lever.co/v0/postings/{company}`
- Ashby: public posting API
- Company list seeded from community-maintained lists (a few hundred companies)

Known coverage gap: companies on Workday are out of scope for v1 (their API is much uglier).

Build plan

Context for scoping: I know C++ and DSA, I am learning Go this week (day plan already set). My IT job and the fall semester start early August, so depth cannot all live at the end.

Weeks	Milestone
1 (now)	Learn Go. Day 7 output is v0: poll 10 hardcoded companies, print new postings to stdout
2–3	Postgres storage, differ, Discord webhook, deployed on a cheap VM (systemd, Nginx, HTTPS)
4	Sponsorship-signal rules, raw-response caching to disk, seed real company list
5	Worker pool + rate limiting, scale company list to hundreds (pulled forward on purpose, see risks)
6	Docker, GitHub Actions auto-deploy
7–8	Prometheus + Grafana, load test the REST API, fix what breaks
9–10	Hard part: scale to thousands of companies, or proper full-text search with ranking

Deadline logic: resume-ready by October 1, 2026, since 2027 applications open August–September.

Non-goals (v1)

- No web UI. Discord only. (Personal rule, 9 weeks minimum.)
- No ML for sponsorship detection. Rules and regex only.
- No Workday ingestion.
- No auth or multi-tenancy until friends actually ask to join.
- Not a startup. This is a resume project with one honest user at launch.

Risks and mitigations

- Unofficial endpoints can change or throttle. Mitigation: cache every raw API response to disk from day one, so development never blocks on upstream, and parser breakage is replayable.
- Depth is back-loaded and August is busy. Mitigation: worker pool moved forward to week 5, so a truncated project still has interview substance.
- Read-side load is tiny (a few friends). Mitigation: honest phrasing everywhere, "load tested to X req/s", never "handles X req/s of traffic".

- Crowded genre. Mitigation: the sponsorship filter and diffs are the differentiators, plus the answer in the section above.

Success criteria

1. From August, I personally find internships through this tool, not by scrolling.
2. It survives 30 minutes of interview grilling: every component has a "what broke and what I did" story.
3. Deployed, monitored, load tested, README, and I can explain every line.

Questions for you (the reviewer)

1. Scope: does the 10-week table above look realistic for someone starting Go this week, or where would you cut?
2. What is missing that would make you say "now THIS is a real backend project"?
3. Is the sponsorship-signal filter enough of a differentiator, or does it need one more sharp feature?
4. Which week 9–10 hard part is more impressive to you: scale-out (thousands of companies, worker pool tuning) or search (full-text plus ranking)?
5. Anything here you would call resume inflation?